



S. J. P. N. TRUST'S
HIRASUGAR INSTITUTE OF TECHNOLOGY, NIDASOSHI

Accredited at 'A+' Grade by NAAC

Programmes Accredited by NBA: CSE & ECE

Department of Computer Science & Engineering

MACHINE LEARNING

Module 4: **Chapter 10: ARTIFICIAL NEURAL NETWORKS (ANN)**

Dr. S. V. Manjaragi

**Asso. Prof. , Dept. of Computer Science & Engg.,
Hirasugar Institute of Technology, Nidasoshi**

Artificial Neural Networks (ANNs)

- Artificial Neural Networks (ANNs) imitate human brain behaviour and the way in which learning happens in a human.
- The human brain constitutes a mass of neurons that are all connected as a network, which is actually a directed graph.
- These neurons are the processing units which receive information, process it and then transmit this data to other neurons that allows humans to learn almost any task.
- **ANN is a learning mechanism that models a human brain to solve any non-linear and complex problem.**
- **Each neuron is modelled as a computing unit, or simply called as a node in ANN, that is capable of doing complex calculations.**
- ANN is a system that consists of many such computing units operating in parallel that can learn from observations.

Applications of ANN

- Natural Language Processing (NLP),
- pattern recognition, face recognition,
- speech recognition, character recognition,
- text processing,
- stock prediction,
- computer vision, etc.
- ANNs also have been considerably used in other engineering fields such as Chemical industry, Medicine, Robotics, Communications, Banking, and Marketing.

Introduction to Neurons

- **The human nervous system has billions of neurons** that are the processing units which make humans to perceive things, to hear, to see and to smell.
- **It is a learning system that consists of functional units called nerve cells, typically called as neurons.**
- The human nervous system is divided into two sections called the **Central Nervous System (CNS) and the Peripheral Nervous System (PNS).**
- The neurons are basically classified into **three types called sensory neurons, motor neurons and interneurons.**
- Sensory neurons get information from different parts of the body and bring it into the CNS
- motor neurons receive information from other neurons and transmit commands to the body parts.
- The CNS consists of only interneurons which connect one neuron to another neuron by receiving information from one neuron and transmitting it to another.

Biological Neurons

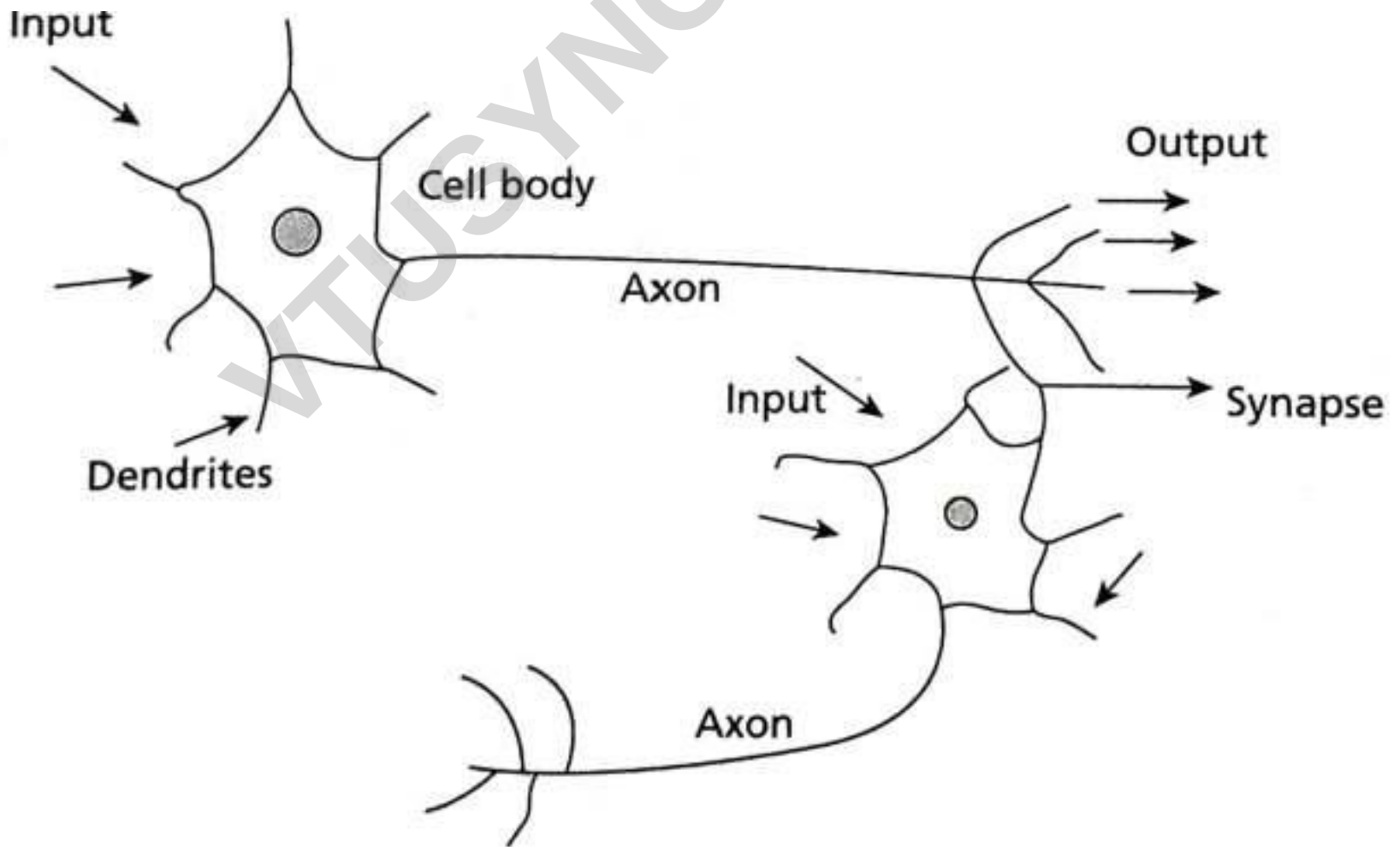


Figure 10.1: A Biological Neuron

Artificial Neurons

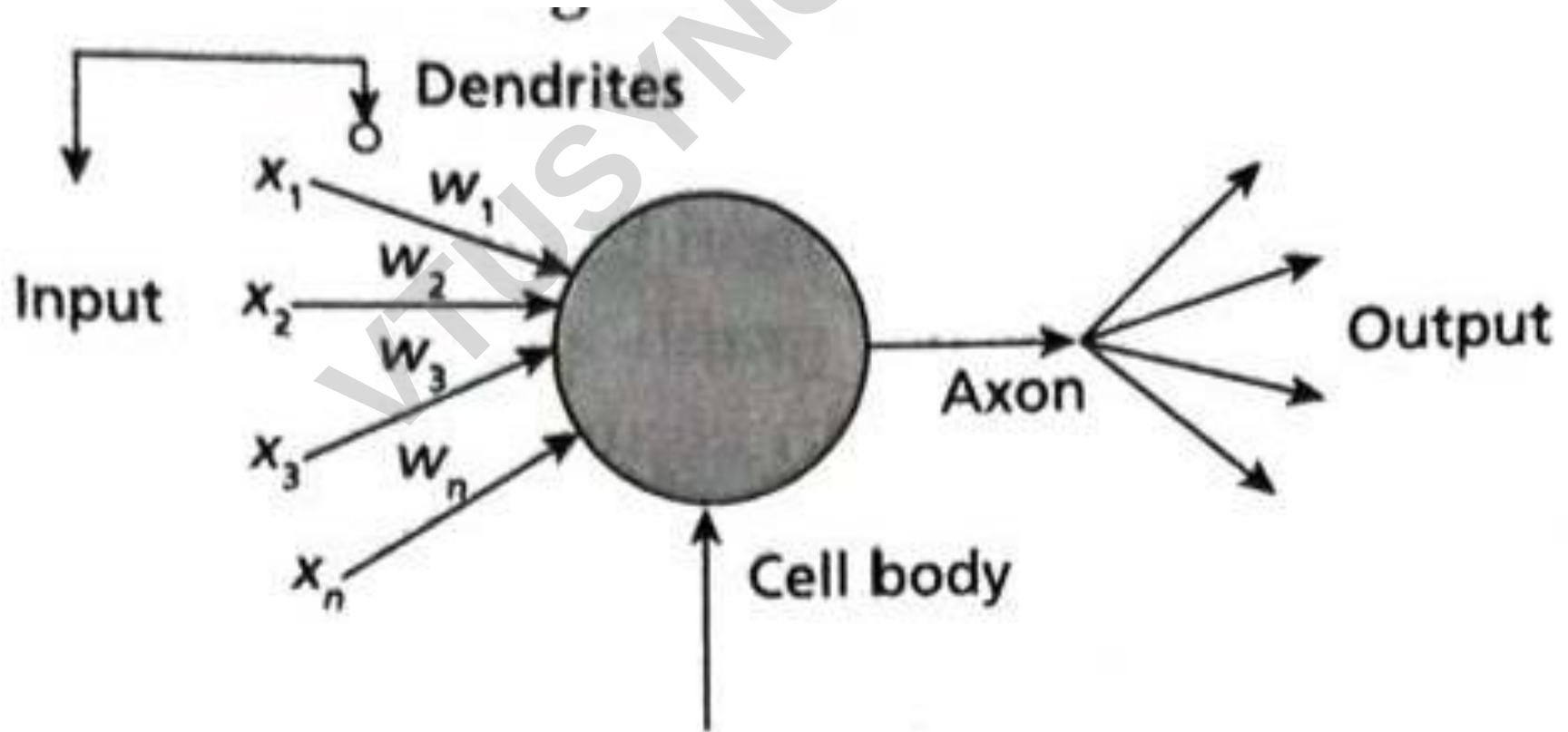


Figure 10.2: An Artificial Neuron

Simple Model of an Artificial Neuron

- The first mathematical model of a biological neuron was designed by McCulloch & Pitts in 1943. **It includes two steps:**
 - It receives weighted inputs from other neurons
 - It operates with a threshold function or activation function
- The received inputs are computed as a weighted sum which is given to the activation function and if the sum exceeds the threshold value the neuron gets fired.
- The mathematical model of a neuron is shown in Figure 10.3.

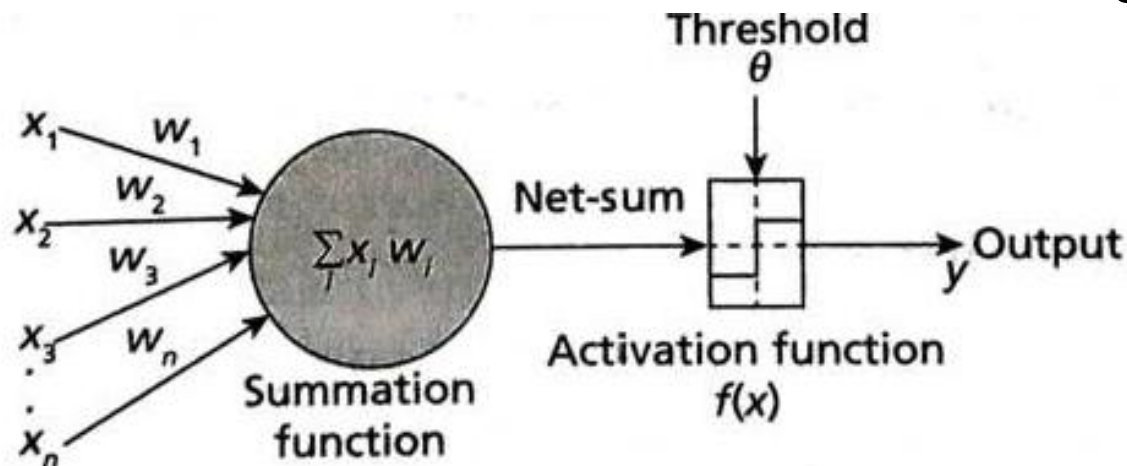


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

Simple Model of an Artificial Neuron

- The neuron is the basic processing unit that receives a set of inputs $x_1, x_2, \dots, x_n$ and their associated weights $w_1, w_2, \dots, w_n$.
- The Summation function 'Net-sum' Eq. (10.1) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad (10.1)$$

- The activation function is a binary step function which outputs a value 1 if the Net-sum is above the threshold value ϑ , and a 0 if the Net-sum is below the threshold value ϑ .
- Therefore, the activation function is applied to Net-sum as shown in Eq. (10.2).

$$f(x) = \text{Activation function (Net - sum)} \quad (10.2)$$

$$\text{Then, output of a neuron } Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.3)$$

Simple Model of an Artificial Neuron

- **McCulloch & Pitts Neuron model can represent only a few Boolean functions.**
- A Boolean function has binary inputs and provides a binary output.
- For example, an AND Boolean function neuron would fire when all the inputs are 1, whereas an OR Boolean function neuron would fire even when one input is 1.
- The weight and threshold values are fixed in this mathematical model.

Artificial Neural Network Structure

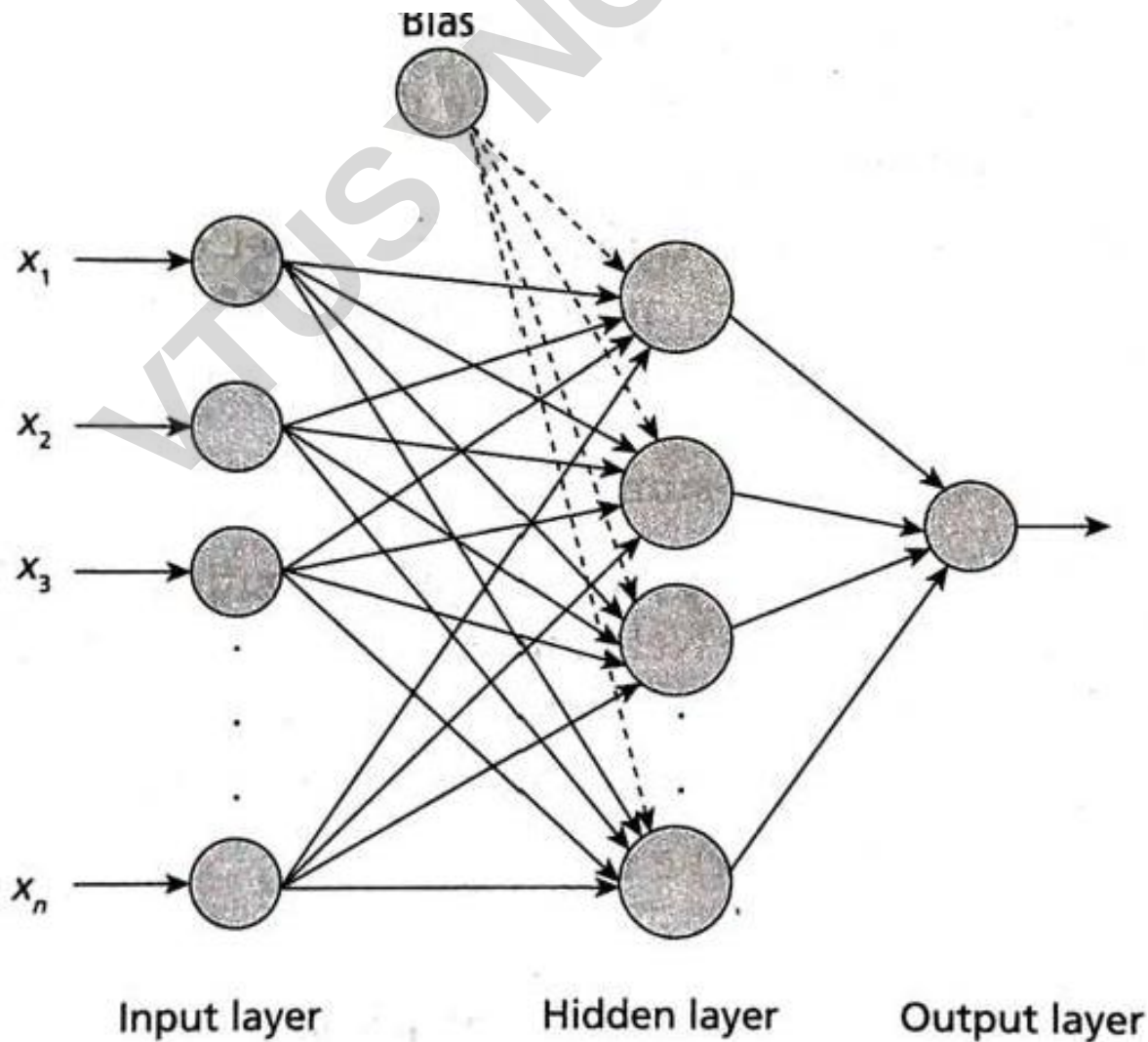


Figure 10.4: Artificial Neural Network Structure

Activation Functions

- **Activation functions are mathematical functions** associated with each neuron in the neural network that map input signals to output signals.
- **It decides whether to fire a neuron or not based on the input signals the neuron receives.**
- These functions normalize **the output value of each neuron either between 0 and 1 or between -1 and $+1$.**
- Typical activation functions can be linear or non-linear.
- **Linear functions** are useful when the input values can be classified into any one of the two groups and are generally **used in binary perceptions.**
- **Non-linear functions, on the other hand, are continuous functions that map the input in the range of $(0, 1)$ or $(-1, 1)$, etc.**
- These functions are useful in learning high-dimensional data or complex data such as audio, video and images.

Activation Functions used in ANN

1. Identity Function or Linear Function

$$f(x) = x \quad \forall x \quad (10.4)$$

The value of $f(x)$ increases linearly or proportionally with the value of x . This function is useful when we do not want to apply any threshold. The output would be just the weighted sum of input values. The output value ranges between $-\infty$ and $+\infty$.

2. Binary Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.5)$$

The output value is binary, i.e., 0 or 1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs 1 or else it outputs 0.

3. Bipolar Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ -1 & \text{if } f(x) < \theta \end{cases} \quad (10.6)$$

The output value is bipolar, i.e., +1 or -1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs +1 or else it outputs -1.

Activation Functions used in ANN

4. Sigmoidal Function or Logistic Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (10.7)$$

It is a widely used non-linear activation function which produces an S-shaped curve and the output values are in the range of 0 and 1. It has a vanishing gradient problem, i.e., no change in the prediction for very low input values and very high input values.

5. Bipolar Sigmoid Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \quad (10.8)$$

It outputs values between -1 and +1.

6. Ramp Functions

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.9)$$

It is a linear function whose upper and lower limits are fixed.

7. Tanh – Hyperbolic Tangent Function

The Tanh function is a scaled version of the sigmoid function which is also non-linear. It also suffers from the vanishing gradient problem. The output values range between -1 and 1.

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (10.10)$$

Activation Functions used in ANN

8. ReLu – Rectified Linear Unit Function

This activation function is a typical function generally used in deep learning neural network models in the hidden layers. It avoids or reduces the vanishing gradient problem. This function outputs a value of 0 for negative input values and works like a linear function if the input values are positive.

$$r(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.11)$$

9. Softmax Function

This is a non-linear function used in the output layer that can handle multiple classes. It calculates the probability of each target class which ranges between 0 and 1. The probability of the input belonging to a particular class is computed by dividing the exponential of the given input value by the sum of the exponential values of all the inputs.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=0}^k e^{x_j}} \text{ where } i = 0 \dots k \quad (10.12)$$

PERCEPTRON AND LEARNING THEORY

- The first neural network model 'Perceptron', designed by Frank Rosenblatt in 1958, is a linear binary classifier used for supervised learning.
- He modified the McCulloch & Pitts Neuron model by **combining two concepts, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights.**
- He introduced variable weight values and an extra input that represents bias to this model.
- He proposed that artificial neurons could actually learn weights and thresholds from data and came up with a supervised learning algorithm that enabled the artificial neurons to learn the correct weights from training data by itself.

PERCEPTRON AND LEARNING THEORY

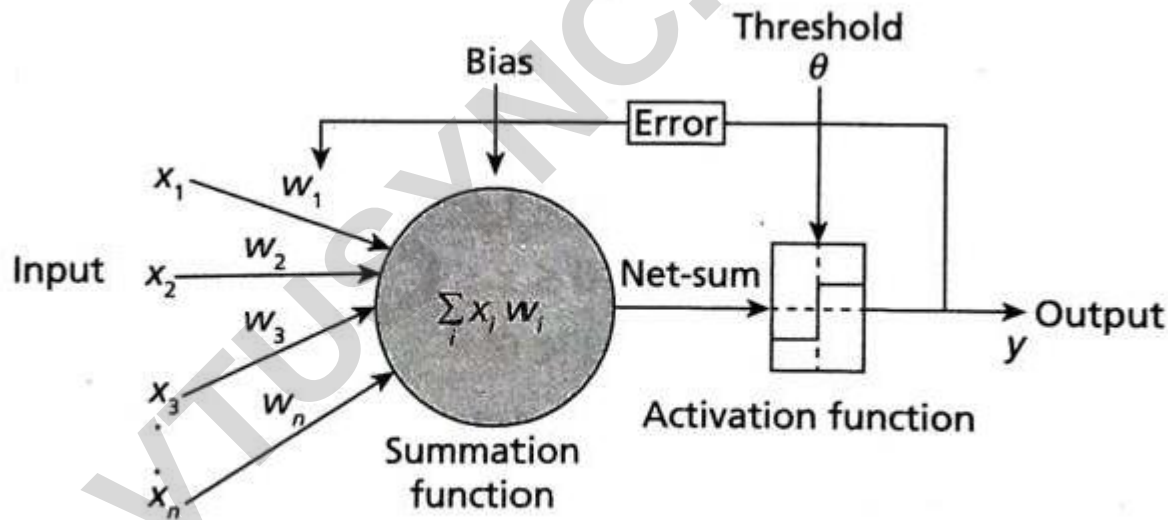


Figure 10.5: Perceptron Model

Consists of 4 steps:

- Inputs from other neurons
- Weights and bias
- Net sum
- Activation function

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i$$

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step}((x_1 w_1 + x_2 w_2) - \theta)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} :

$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y .]

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

Types of ANN

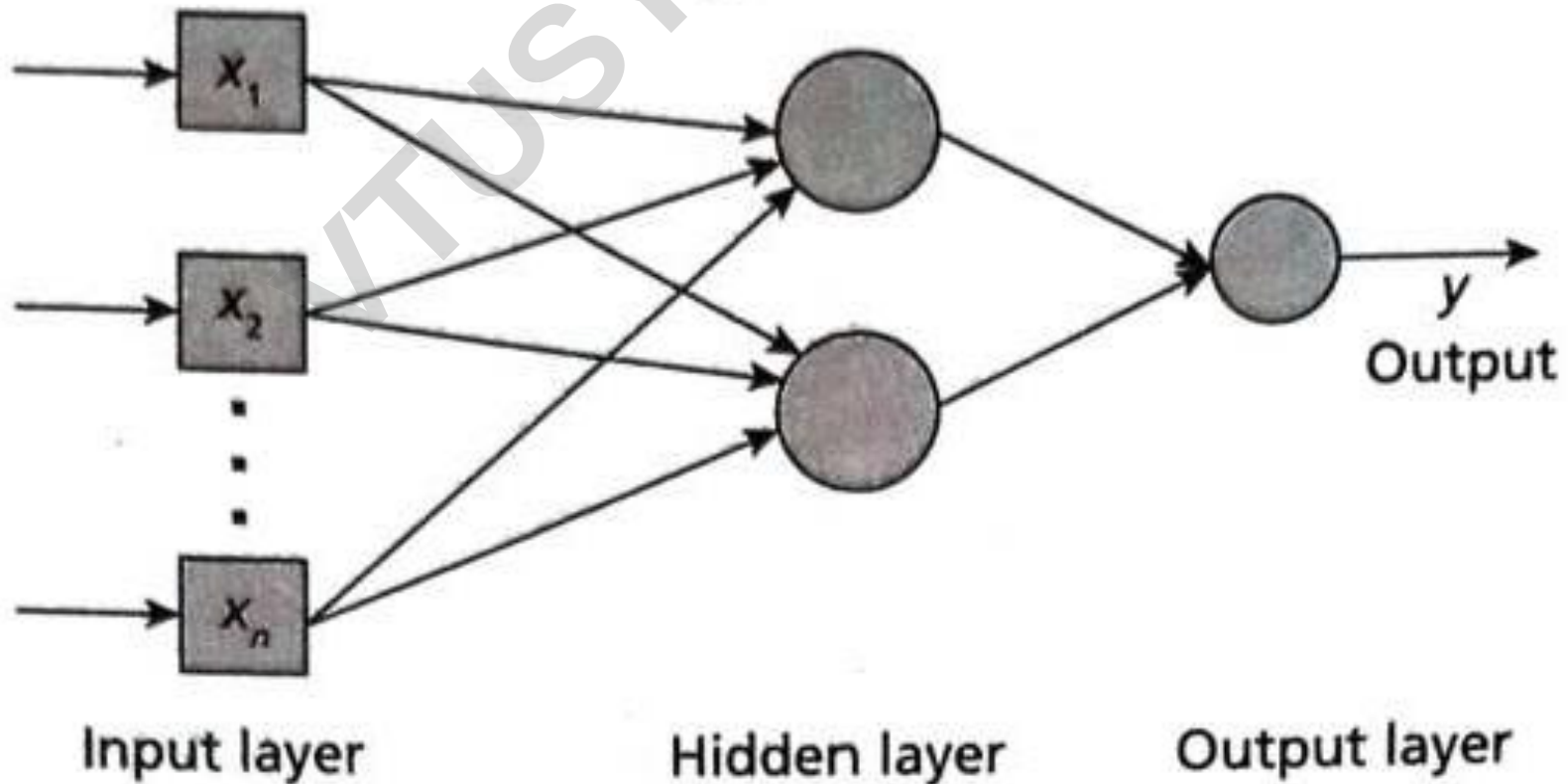


Figure 10.7: Model of a Feed Forward Neural Network

Types of ANN

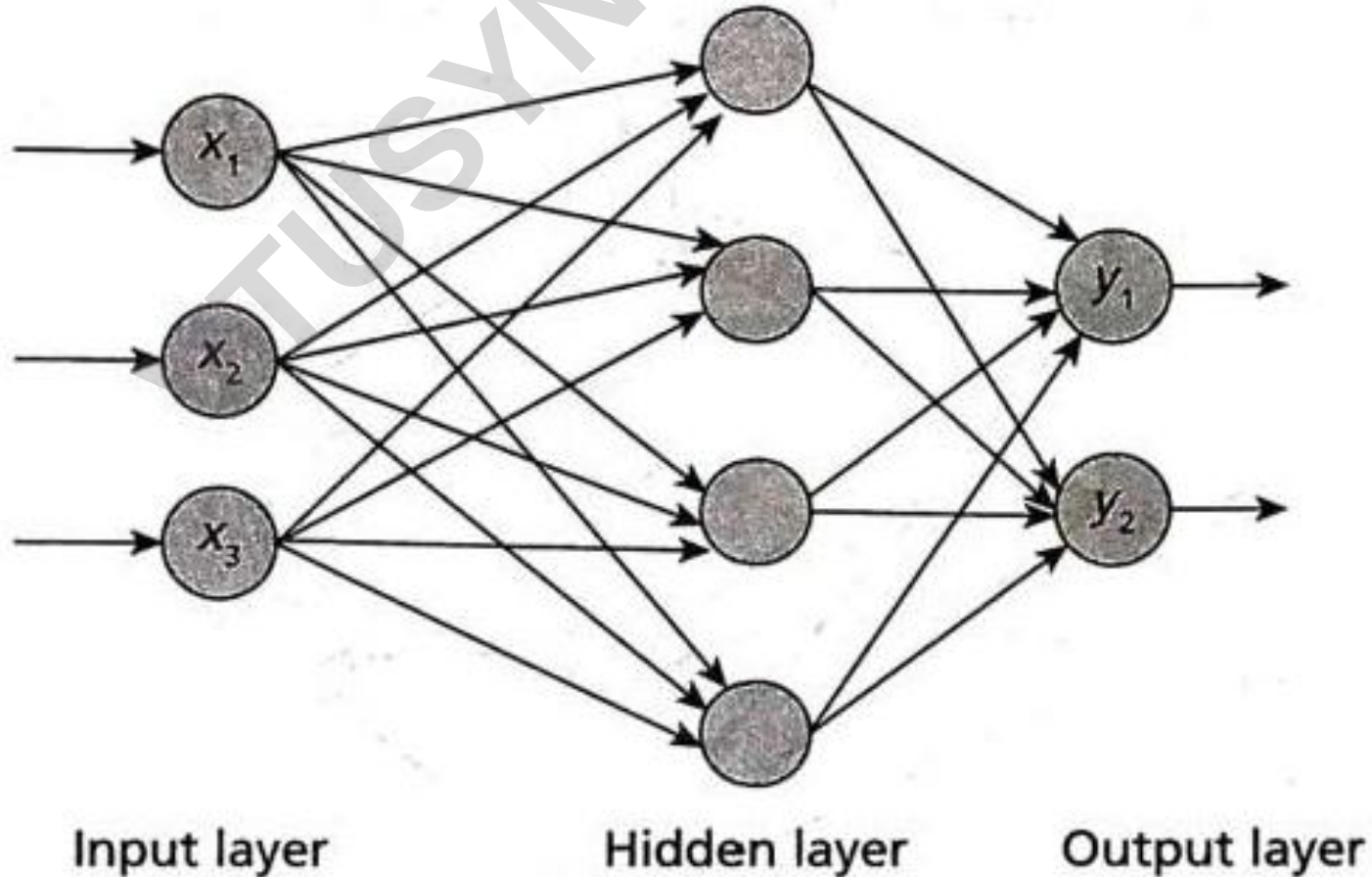


Figure 10.8: Model of a Fully Connected Neural Network

Types of ANN

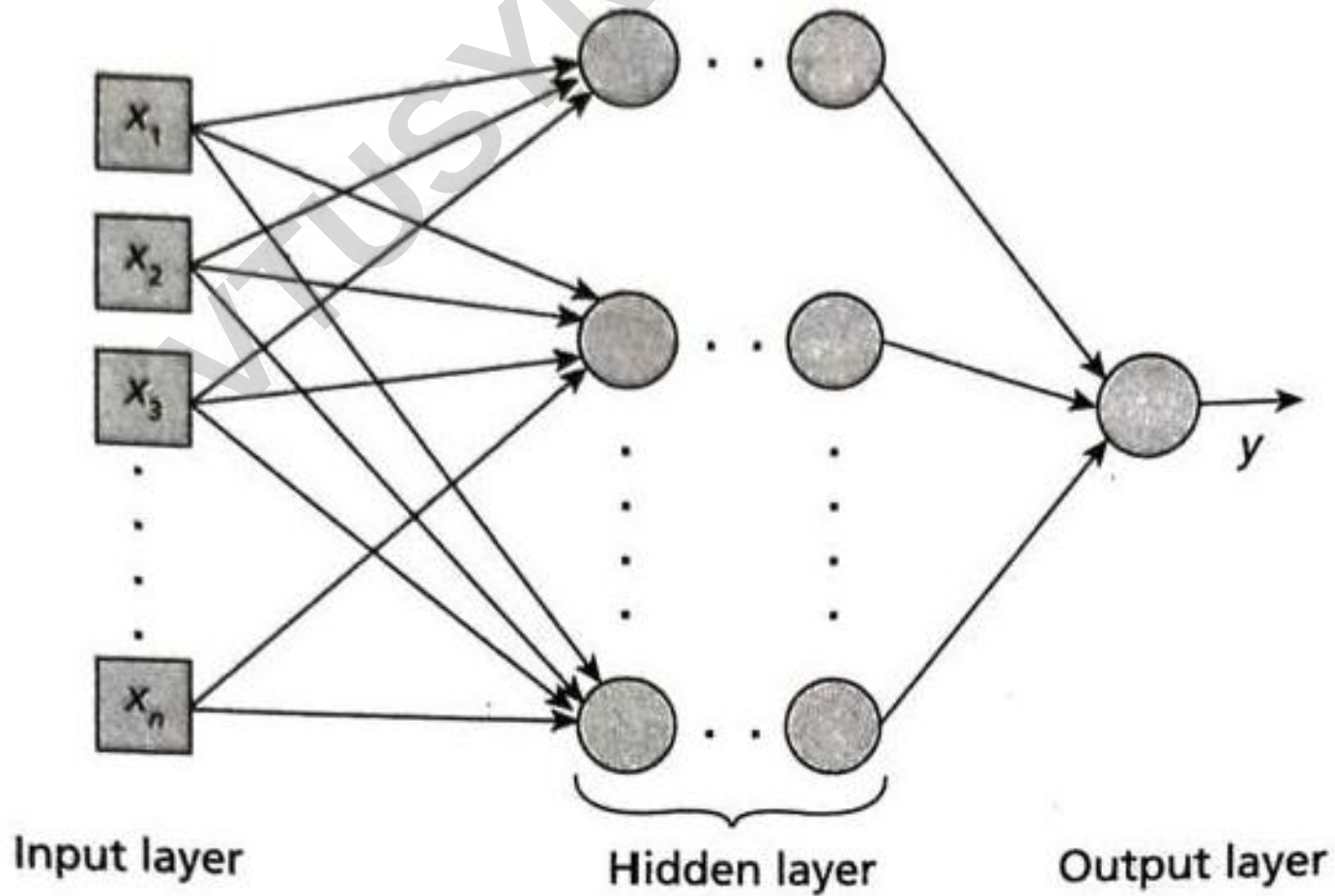


Figure 10.9: Model of a Multi-Layer Perceptron

Types of ANN

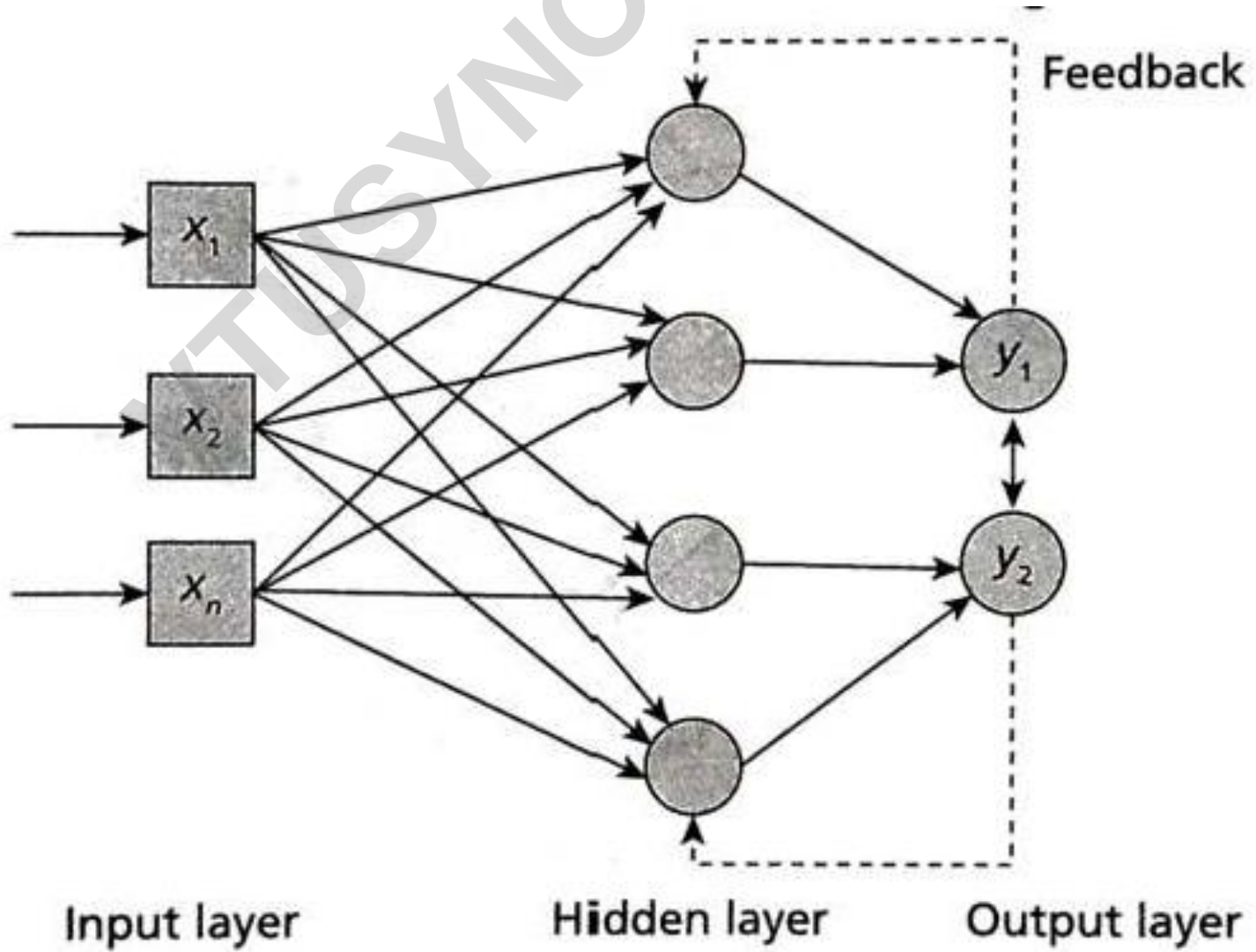


Figure 10.10: Model of a Feedback Neural Network

POPULAR APPLICATIONS OF ARTIFICIAL NEURAL NETWORKS

1. **Real-time applications:** Face recognition, emotion detection, self-driving cars, navigation systems, routing systems, target tracking, vehicle scheduling, etc.
2. **Business applications:** Stock trading, sales forecasting, customer behaviour modelling, Market research and analysis, etc.
3. **Banking and Finance:** Credit and loan forecasting, fraud and risk evaluation, currency price prediction, real-estate appraisal, etc.
4. **Education:** Adaptive learning software, student performance modelling, etc.
5. **Healthcare:** Medical diagnosis or mapping symptoms to a medical case, image interpretation and pattern recognition, drug discovery, etc.
6. **Other Engineering Applications:** Robotics, aerospace, electronics, manufacturing, communications, chemical analysis, food research, etc.

Advantages of ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations of ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.